

Tourism Government — Architectural Module Mastery & Verbal Presentation Manual

Complete Technical Specifications and 5-Minute Script for Dashboard, Notification, and Reporting Systems

■ SENIOR-LEVEL TECHNICAL MANUAL:

This architectural guide is dedicated **strictly to your three modules** (Dashboard, Notification, and Reporting). It provides absolute, deep descriptions of the frontend (React, hooks, Tailwind, Framer Motion) and backend (Spring Cloud, OpenFeign, Resilience4j configuration, Mockito unit testing) systems. It details the step-by-step end-to-end data flows and wraps up with the ultimate word-for-word 5-minute presentation script.

Chapter 1: Frontend Architecture & Technology Implementations

Our React frontend runs on Vite for instant compile-time execution. It leverages the following technologies to create a premium, state-of-the-art UI:

1. React Hooks & Global Contexts:

- `useState`: Local state handles user inputs in our Report download panels and toggles modal overlays.
- `useEffect`: Triggers on-load API requests using `Axios` to aggregate Dashboard counts, and manages a background polling interval to count unread notifications.
- `useContext` (`AuthContext` / `NotificationContext`): Handles global states. The token is parsed from `localStorage`, and a central context keeps the red bell unread count synchronized across every route.

2. Tailwind CSS Design System:

- Utilizes modern utility classes to build a modern glassmorphism UI: `bg-white bg-opacity-20 backdrop-blur-md border border-white border-opacity-30 shadow-lg`.
- Responsive layouts are configured using utility grids (e.g. `grid grid-cols-1 md:grid-cols-3 gap-6`) for seamless mobile-to-desktop resizing.

3. Framer Motion Micro-Animations:

- Standard elements are replaced with animated motion counterparts, e.g. `<motion.div>`.
- Dashboard cards load with a elegant fade-and-rise transition:
`initial={{ opacity: 0, y: 15 }} animate={{ opacity: 1, y: 0 }} transition={{ duration: 0.4 }}`.
- Notification bells scale up dynamically on hover to create subtle, engaging feedback.

Chapter 2: Backend Architecture & Resilience Implementations

The backend follows a pure Spring Boot microservices structure, communicating via a centralized **Spring Cloud API Gateway**:

1. Isolated Database-per-Service Model:

- The Notification service stores records in `notificationdb`, while the Reporting service records audits in `reportdb`, completely avoiding cross-schema joins.

2. Feign Client Discovery with Eureka:

- Interfaces (like `UserClient`) utilize the `@FeignClient` annotation to let Spring Cloud fetch host details directly from Eureka, bypassing hardcoded IPs.

3. Feign-Level Resilience4j Circuit Breaker Configurations:

- Configured globally in `application.properties` using these 5 crucial properties:
 - `spring.cloud.openfeign.circuitbreaker.enabled=true` (Binds interceptors to Feign calls)
 - `slidingWindowSize=10` (Tracks outcomes of the last 10 requests)
 - `failureRateThreshold=50` (Trips to OPEN if 50% or more calls fail)
 - `waitDurationInOpenState=10000` (Keeps circuit open for 10 seconds to allow target recovery)
 - `permittedNumberOfCallsInHalfOpenState=5` (Allows 5 trial requests. At least 3 must succeed to close the circuit; if 3 or more fail, it trips back to OPEN).

Chapter 3: Deep End-to-End Data Flows (Your Modules)

1. Dynamic Dashboard Render Flow (Frontend-to-Backend)

- **Step 1:** The user loads the dashboard. The React `useEffect` hook fires off an Axios GET request to our Gateway `http://localhost:8080/api/dashboard/stats`. The gateway checks the JWT token in the header.
- **Step 2:** The gateway forwards the request to the target services. The controller maps it, coordinates with JpaRepositories, and aggregates active site counts, scheduled event schedules, and safety compliance levels.
- **Step 3:** The controller replies with an aggregated JSON payload. React catches this response, updates local state hooks, and Framer Motion dynamically animates the dashboard cards on screen.

2. Real-Time Notification Observer Flow (Service-to-Service)

- **Step 1:** An officer registers a new Heritage Site or Schedules an Event. The Site Service triggers a background REST POST call to our Notification Service.
- **Step 2:** The `NotificationController` catches the request, executes validation DTO rules, persists the log in `notificationdb`, and fires a background HTML email alert using `JavaMailSender`.
- **Step 3:** On the React UI, our global `NotificationContext` performs background polling. It queries the backend endpoint, detects the new unread flag, and immediately updates the red unread count badge next to the notification bell icon.

3. High-Security Administrative Reporting Flow (Security-to-Aggregation)

- **Step 1:** An administrator submits a report compile request. The React panel posts a request with a valid JWT token to `http://localhost:8080/api/reports/generate`.
- **Step 2:** The `ReportController` intercepts the DTO. The service uses Feign Client to call the User Service and verify the requester's role. If the role is `TOURIST`, it immediately throws an HTTP 403 Forbidden exception, blocking access.
- **Step 3:** If authorized, the service queries 5 separate microservices (Site, Event, Compliance, Notification, Program) via Feign clients. It aggregates the data, compiles a database entry in `reportdb`, prints a structural report document, and sends it as a base64 download stream to the React UI.

Chapter 4: The Ultimate 5-Minute Spoken Verbal Script

Deliver this script with steady pacing, high confidence, and clear voice transitions to showcase your senior-level architecture expertise:

■ Phase 1: Intro & High-Level Architecture (0:00 - 1:00)

"Good morning. I am thrilled to present my core modules in the Tourism Government platform. I was responsible for designing, developing, and testing three core systems: the frontend Admin Dashboard, the backend Notification Service, and the administrative Reporting Service."

"On the Frontend, we used React Vite for dynamic, hot-reloading speed. We implemented Tailwind CSS to build a beautiful glassmorphism dashboard, and integrated Framer Motion to create engaging micro-animations for our components. On the Backend, we built independent Spring Boot microservices following a strict Database-per-Service model. We routed all REST requests through a single Spring Cloud API Gateway with JWT authentication, and registered our instances dynamically on a Eureka discovery server."

■ Phase 2: Notification System & Fault Tolerance (1:00 - 2:30)

"Let's deep dive into the Notification module. This service listens for critical actions like site creation or regulatory violations. It stores logs in its own isolated database schema, notificationdb, to prevent any table locking across our microservices. It also triggers dynamic HTML emails to keep officers informed."

"To handle network faults, we enabled Feign-level Circuit Breakers with Resilience4j. Our configuration monitors a sliding window of the last 10 requests. If 5 or more fail, the circuit transitions to OPEN, blocking calls and routing requests to a fallback class returning safe defaults. Once the 10-second cooldown expires, the circuit goes to HALF-OPEN, running 5 trial requests. At least 3 out of 5 must succeed to safely close the circuit. On the React UI, a global NotificationContext manages background polling, dynamically updating the unread bell count badge in real-time."

■ Phase 3: Reporting Aggregation & Security (2:30 - 4:00)

"Now, let's look at the Reporting module. This is a high-security service. When a user requests a custom report, the service calls our User Feign Client to verify their security role. If a Tourist attempts this, our system throws an HTTP 403 Forbidden exception, blocking the action."

"If the requester is an Admin, the Reporting Service acts as an aggregator. It queries 5 separate downstream microservices via Feign, processes the data, saves an audit trace to reportdb, and compiles the document into a base64 download stream for our React UI. To verify this complex logic without database dependencies, we implemented robust Mockito unit tests, mocking out JpaRepositories and Feign clients to validate happy paths and security boundaries in milliseconds!"

■ Phase 4: Conclusion & System Impact (4:00 - 5:00)

"By combining isolated database schemas, robust Feign fallbacks, sliding window circuit breakers, and Mockito testing, we built an architecture that is highly modular, secure, and resilient, guaranteeing 100% operational uptime for our government administrators. Thank you so much, and I welcome any questions!"